Ereditarietà e polimorfismo

Introduzione

Nelle lezioni precedenti abbiamo approfondito il concetto di **classe** e di **oggetto**, evidenziando l'importanza dell'**incapsulamento** per proteggere lo stato interno di un oggetto, e l'uso dei **modificatori di accesso** per controllarne la visibilità. Abbiamo inoltre visto come gli **attributi statici** e i **metodi statici** consentano di condividere dati e comportamenti a livello di classe, e come le **inner class** possano essere utilizzate per raggruppare logicamente classi strettamente correlate.

Partendo da questi concetti, la lezione di oggi introduce **l'ereditarietà**, uno dei pilastri della programmazione orientata agli oggetti.
L'ereditarietà permette di:

- **creare nuove classi a partire da classi esistenti, riutilizzandone attributi e metodi e specializzandoli secondo necessità.**
  Questo apre la strada al **polimorfismo:**
- **cioè alla capacità di trattare oggetti di classi diverse in modo uniforme e di far invocare automaticamente il comportamento corretto in base alla classe reale dell'oggetto.**
  Alla base di questo meccanismo ci sono due concetti chiave della JVM: il **binding dinamico**, che determina a run-time quale metodo invocare, e la **Virtual Method Invocation**, che realizza concretamente l'invocazione del metodo corretto.

Questi concetti consentono di scrivere codice più flessibile, modulare e facilmente estendibile, permettendo di aggiungere nuove classi e comportamenti senza modificare il codice già esistente.

Concetto di ereditarietà

L'**ereditarietà** è uno dei punti di forza principali di Java e, più in generale, della programmazione orientata agli oggetti (OOP).

Essa permette di creare nuove classi partendo da classi già esistenti, riutilizzando codice e strutturando meglio il dominio dell'applicazione.
In pratica, una classe **figlia** eredita attributi e metodi dalla classe **padre**, e può aggiungerne di nuovi o ridefinirne alcuni.

I principali vantaggi dell'ereditarietà sono:

1. **Massimizzare il riutilizzo del codice**
   - **Separando gli aspetti generici da quelli specifici, possiamo definire classi base con funzionalità comuni (es. `Persona`) e poi creare sottoclassi più specializzate (es. `Studente` o `Docente`).**
   - Questo approccio ci permette di organizzare gli elementi del dominio in maniera logica: **gli elementi generici in classi base, gli aspetti specifici nelle sottoclassi.**
   - In questo modo, ogni classe si concentra sul proprio ambito di responsabilità.
2. **Facilità di evoluzione e manutenzione**
   - **L'aggiunta di nuove funzionalità avviene estendendo le classi esistenti, senza modificare il codice già testato.**
   - **Questo riduce i rischi di introdurre errori e rende il software più manutenibile e scalabile.**

Definizione di ereditarietà

L'**ereditarietà** permette di creare nuove classi che **riutilizzano**, **estendono** e, se necessario, **modificano** (tramite overriding) il comportamento di altre classi già esistenti.

- La classe i cui membri (attributi e metodi) vengono ereditati è detta **classe base** o **classe padre**.
- La classe che eredita questi membri è detta **classe derivata** o **classe figlia**.

Si parla di **superclasse** perché nei diagrammi UML viene rappresentata **sopra** alla sottoclasse.

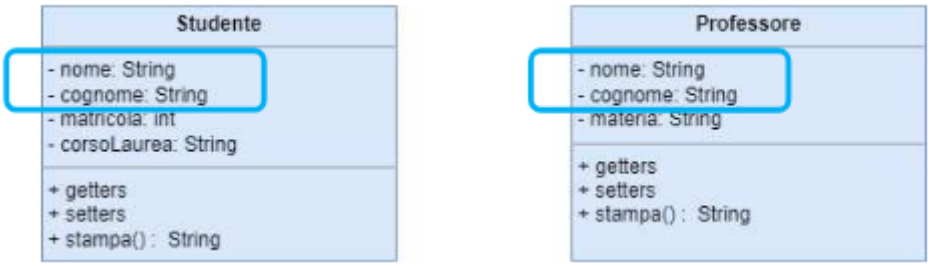
Questa gerarchia può estendersi ulteriormente:

- una **classe figlia** può a sua volta diventare **classe padre** per altre classi, creando una **catena di ereditarietà**.
- In questo modo, possiamo costruire strutture a più livelli, dove ogni livello aggiunge funzionalità specifiche senza duplicare il codice comune.

Esempio 1 : dividere apsetti generali e specifici

Supponiamo di voler modellare alcune entità del **dominio universitario**, come **studenti** e **professori**:

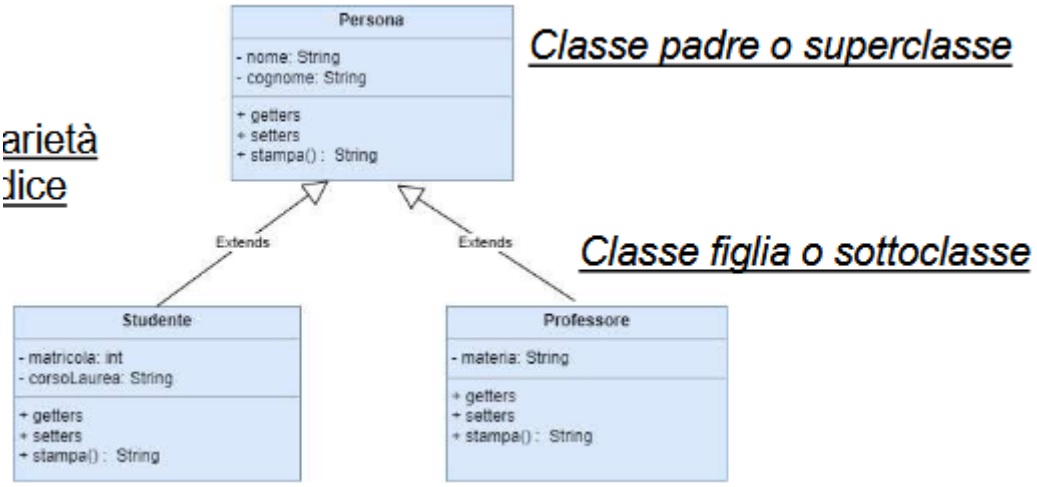
- **In comune** hanno: `nome` e `cognome`.
- **Specifico per lo studente**: `matricola` e `corso di laurea`.
- **Specifico per il professore**: `materia insegnata`.



Se creassimo le due classi separatamente, copiando e incollando tutti i campi comuni, avremmo **ridondanza di dati**.

> ⚠ **Il copia-incolla non è riuso del codice: aumenta il rischio di errori e rende difficile la manutenzione.**

Per ottimizzare, possiamo creare una **classe più generale**, chiamata `Persona`, che contenga **attributi e metodi comuni**.
Le classi `Studente` e `Professore` diventeranno **sottoclassi** di `Persona`:



- **Ereditano** attributi e metodi della superclasse (`nome`, `cognome`, eventuali metodi comuni).
- **Aggiungono** attributi e comportamenti specifici (`matricola`, `corso di laurea`, `materia`).
- **Possono modificare** (override) comportamenti ereditati se necessario.

> ✓ **In questo modo:**
>
> - Si riduce la **ridondanza**.

- Si favorisce il **riuso del codice**.
- Si ottiene una struttura più **chiara e manutenibile**.

Esempio 2: evolvere strutture e comportamenti

Supponiamo ora di avere una classe `Impiegato` che modella il **dipendente di un'azienda**.
Questa classe rappresenta correttamente la situazione iniziale del dominio applicativo.

Con il tempo, però, nasce una nuova esigenza: **modellare i manager**, che sono una **specializzazione** degli impiegati.
Un **manager**:

- è a tutti gli effetti un **impiegato**
- possiede **caratteristiche aggiuntive**, come ad esempio una **segretaria**
- prevede una **logica diversa** per l'aumento di stipendio, quindi una diversa implementazione del metodo `incrementaSalario()`

A questo punto, dal punto di vista progettuale, si potrebbero ipotizzare diverse soluzioni.

Soluzioni non ottimali

1. **Copiare e incollare** la classe `Impiegato` per creare la classe `Manager`
   → soluzione scorretta: **introduce duplicazione di codice e problemi di manutenzione.**
2. **Fondere le due classi** introducendo attributi aggiuntivi (boolean, stringhe, enum)
   → soluzione poco elegante: **la classe diventa più complessa e perde chiarezza semantica.**

Entrambe le soluzioni **violano i principi della OOP** e rendono il codice difficile da evolvere.

Soluzione corretta: usare l'ereditarietà

La soluzione migliore consiste nel creare una nuova classe `Manager` come **evoluzione della classe** `Impiegato`.

In questo modo:

- `Manager` **estende** `Impiegato`
- **eredita tutti gli attributi e i metodi comuni**
- **aggiunge i propri membri specifici (es.** `segretaria` **)**
- **ridefinisce il metodo** `incrementaSalario()` **per adattarlo alle proprie esigenze**

In termini OOP:

- `Impiegato` **è la classe padre (o superclasse)**
- `Manager` **è la classe figlia (o sottoclasse)**

La ridefinizione del metodo `incrementaSalario()` è un esempio di **overriding**:

- **la sottoclasse fornisce una nuova implementazione di un metodo già definito nella superclasse.**

> Si può pensare all'overriding come a un aggiornamento software:
> **la nuova versione sovrascrive quella precedente, mantenendo però lo stesso "nome" e la stessa interfaccia**.

Nota progettuale

Dal punto di vista del dominio, una segretaria potrebbe essere modellata come una **classe autonoma** (e potenzialmente come una sottoclasse di `Impiegato`).
Per semplicità, in questo esempio la segretaria viene rappresentata come **attributo del** `Manager`, ad esempio di tipo `String`.

Generalizzazioni UML

Nel contesto della modellazione UML, il concetto di **ereditarietà** viene espresso tramite la **relazione di generalizzazione**.

La **generalizzazione** rappresenta una relazione gerarchica tra classi, in cui:

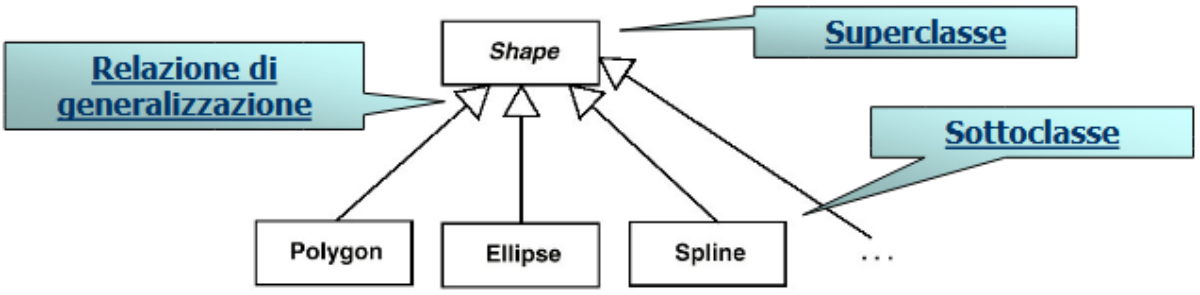
- la **classe di livello superiore** (detta **superclasse**) contiene **gli elementi comuni a più entità del dominio**;
- le **classi di livello inferiore** (dette **sottoclassi**) rappresentano **specializzazioni della superclasse e contengono gli aspetti specifici**.

In altre parole, attraverso la generalizzazione si realizza lo stesso principio visto a livello di codice Java:

- **gli attributi e i comportamenti generali vengono definiti una sola volta nella superclasse, mentre le sottoclassi si limitano ad aggiungere o specializzare ciò che le distingue.**

Dal punto di vista grafico, UML utilizza una **notazione standard** per rappresentare la generalizzazione:

- **una linea continua che collega la sottoclasse alla superclasse**;
- **un triangolo vuoto orientato verso la classe più generale**.



Il triangolo indica chiaramente che la relazione è del tipo *"è un"* (is_a): una sottoclasse **è un tipo particolare** della superclasse.

> 💡 Ad esempio:
> un `Manager` **è un** `Impiegato`,
> uno `Studente` **è una** `Persona`.

La generalizzazione consente quindi di:

- **strutturare il dominio in modo gerarchico e ordinato;**
- **evitare la duplicazione delle informazioni;**
- **facilitare l'estensione futura del modello.**

Questo meccanismo costituisce la base sia dell'**ereditarietà** sia del **polimorfismo**, che a livello UML viene reso possibile proprio grazie alla relazione di generalizzazione.

Relazione *is-a* e sottocategorie

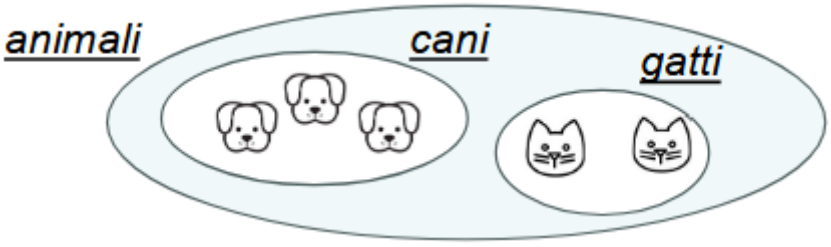
La relazione tra una **classe** e le sue **sottoclassi** è detta relazione **_is-a_** (*è-un*). **Questa relazione induce a interpretare le classi come insiemi di oggetti e le sottoclassi come sottoinsiemi più specifici**.

In termini concettuali:

- **la superclasse rappresenta l'insieme più generale**;
- **le sottoclassi rappresentano sottoinsiemi che condividono le caratteristiche comuni, ma introducono elementi distintivi**.

Ad esempio, se `Cane` e `Gatto` sono sottoclassi di `Animale`, allora:

- un **cane è un animale**;
- un **gatto è un animale**.



Ogni oggetto di tipo `Cane` o `Gatto` appartiene quindi **contemporaneamente** a più categorie:

- alla categoria specifica (`Cane` o `Gatto`);
- alla categoria più generale (`Animale`).

Questa osservazione ci conduce direttamente al concetto di **polimorfismo intrinseco degli oggetti**.

Un oggetto, infatti, **non appartiene a una sola categoria**, ma può essere visto e trattato come istanza di:

- una classe specifica;
- una o più classi più generali nella gerarchia di ereditarietà.

In Java, questo significa che **un oggetto può essere referenziato tramite il tipo della superclasse**, mantenendo comunque il proprio comportamento specifico.

È proprio questa appartenenza a più categorie che rende possibile il **polimorfismo:**

- **cioè la capacità di utilizzare oggetti diversi attraverso un'unica interfaccia comune, lasciando che sia il loro tipo reale a determinare il comportamento effettivo**.

Nel prossimo passaggio, questo concetto verrà reso concreto analizzando il **polimorfismo a livello di codice**, tramite riferimenti di tipo superclasse e metodi sovrascritti.

Definizione di una sottoclasse in Java

Dal punto di vista **sintattico**, in Java una **sottoclasse** viene definita utilizzando la parola chiave `extends`.

La forma generale è la seguente:

Java

```java
public class SottoClasse extends SuperClasse {
    ...
}
```

La keyword `extends` indica esplicitamente che la nuova classe:

- **eredita attributi e metodi dalla superclasse;**
- **specializza il comportamento di una classe già esistente;**
- **rappresenta un concetto più specifico rispetto alla classe padre.**

In altre parole, tramite `extends` stiamo dichiarando una relazione di **ereditarietà** che riflette la relazione concettuale ***is-a*** vista a livello UML.

Collegamento con il concetto di generalizzazione

Quando scriviamo:

Java

```java
public class Manager extends Impiegato {
    ...
}
```

stiamo affermando che:

- **un Manager è un Impiegato (*Manager is-a Impiegato*);**
- **ogni oggetto `Manager` appartiene sia alla categoria `Manager` sia alla categoria più generale `Impiegato`.**

Questo rispecchia esattamente la **generalizzazione UML**:

- `Impiegato` è la **superclasse**, **che contiene gli elementi comuni a tutti i dipendenti;**
- `Manager` è la **sottoclasse**, **che introduce caratteristiche e comportamenti più specifici.**

Effetti dell'uso di `extends`

Definendo una classe come sottoclasse di un'altra:

- **non si duplicano i campi e i metodi della superclasse;**
- si ottiene **riuso del codice**;
- la sottoclasse può:
  - **aggiungere nuovi attributi e metodi;**
  - **ridefinire (override) metodi ereditati per adattarli alle proprie esigenze.**

Nel caso di `Manager`, ad esempio:

- **eredita tutti i membri di `Impiegato`;**
- **aggiunge l'informazione relativa alla segretaria;**
- **fornisce una nuova implementazione del metodo `incrementaSalario`**.

Collegamento con il polimorfismo

L'uso di `extends` non serve solo a riutilizzare codice, ma è anche il **presupposto tecnico del polimorfismo**.

Poiché `Manager` è una sottoclasse di `Impiegato`, è possibile:

- **trattare un oggetto `Manager` come un `Impiegato`;**
- **invocare metodi comuni tramite riferimenti alla superclasse;**
- **lasciare che sia il tipo reale dell'oggetto a determinare il comportamento effettivo (binding dinamico).**

Questo aspetto verrà approfondito quando si analizzerà il **polimorfismo in Java a livello di codice**.

Costruttori ed ereditarietà

Quando una classe estende un'altra classe, **i costruttori NON vengono ereditati**.
Questo è un punto fondamentale dell'ereditarietà in Java:

- **anche se una sottoclasse eredita attributi e metodi dalla superclasse, la responsabilità dell'inizializzazione resta separata.**

Tuttavia, affinché un oggetto della sottoclasse sia costruito correttamente, **la parte di stato ereditata dalla superclasse deve comunque essere inizializzata**.
Per questo motivo, **il costruttore della superclasse deve essere esplicitamente richiamato nel costruttore della sottoclasse**.

La keyword `super`

Come in Python anche Java mette a disposizione la keyword `super`, che consente di richiamare:

- **il costruttore della superclasse**
- **oppure i metodi della superclasse (quando sovrascritti)**

Nel caso dei costruttori, la sintassi è:

Java

```java
super(listaDiParametri);
```

Regola fondamentale:

La chiamata a `super(...)` **deve essere la prima istruzione** del costruttore della sottoclasse.

Questo perché Java deve inizializzare prima la parte "generica" dell'oggetto (superclasse) e solo dopo quella specifica (sottoclasse).

Esempio: `Manager` che estende `Impiegato`

Supponiamo che la classe `Impiegato` abbia il seguente costruttore:

Java

```java
public Impiegato(String nome, double salario, Date dataAssunzione) {
    this.nome = nome;
    this.salario = salario;
    this.dataAssunzione = dataAssunzione;
}
```

La classe `Manager`, che estende `Impiegato`, introduce un nuovo attributo:

Java

```java
private String secretaryName;
```

Il costruttore della sottoclasse dovrà quindi:

1. **inizializzare la parte** `Impiegato` tramite `super(...)`
2. **inizializzare gli attributi specifici di** `Manager`

Java

```java
public class Manager extends Impiegato {

    private String secretaryName;

    public Manager(String n, double s, Date d, String ns) {
        super(n, s, d);          // inizializzazione della superclasse
        this.secretaryName = ns; // inizializzazione specifica
    }
}
```

In questo modo:

- `nome`, `salario` e `dataAssunzione` vengono inizializzati dal costruttore di `Impiegato`;
- `secretaryName` viene inizializzato dal costruttore di `Manager`.

Costruttore di default e chiamata implicita a `super()`

Come abbiamo già specificato tante volte, se **non viene definito alcun costruttore** in una classe, il compilatore Java genera automaticamente un **costruttore di default**, che:

- **non prende parametri;**
- **inizializza gli attributi con i valori di default;**
- **invoca implicitamente `super()` nel caso delle classi che ereditate o che ereditano.**

Nel caso della classe `Manager`, sarebbe come scrivere:

Java

```java
public Manager() {
    super(); // chiamata implicita al costruttore della superclasse
    this.secretaryName = null;
}
```

⚠ Attenzione

Questa chiamata implicita a `super()` **funziona solo se la superclasse possiede un costruttore senza parametri**.

Quindi:

✓ ✅ **se `Impiegato` ha un costruttore `Impiegato()`, tutto compila;**

⚠ ❌ **se `Impiegato` non ha un costruttore a zero parametri, il compilatore segnala errore.**

In tal caso, il programmatore è obbligato a:

- **definire esplicitamente un costruttore nella sottoclasse;**
- **richiamare uno dei costruttori disponibili della superclasse tramite `super(...)`.**

🔗 Collegamento concettuale

Questo meccanismo rafforza due principi chiave dell'OOP:

- **responsabilità chiara**: **ogni classe inizializza il proprio stato;**
- **coerenza dell'oggetto**: **un oggetto non può esistere senza che tutte le sue parti (superclasse e sottoclasse) siano correttamente**

inizializzate

Classe `Manager` completa

Java

```java
public class Manager extends Impiegato{
    private String segretaria;
    public Manager (String nome, Date dataAsssunzione, double salario, String segretaria){
        // questi sono gli attributi ereditati da Impiegato
        super(nome, dataAssunzione, salario);
        this.segretaria = segretaria;

    }
    // aggiunta di metodi nuovi
    public String getSegretaria(){
        return segretaria;
    }
    public void setSegretaria(String segretaria){
        this.segretaria = segretaria;
    }
    //aggiornamento di un comportamento(metodo)→ OVERIDDING
    public void incrementaSalario(double incremento){
        // calcolo bonus
        Date oggi = new Date();
        double bonus = 0.5*(oggi.getYear()+1900 - this.getAnnoAssunzione()); // riuso del getter di Impiegato
        //aggiungo incremento + bonus al salario corrente
        super.incrementaSalario(incremento + bonus); // riuso
    }
    public String toString(){
        return "Manager: " + super.toString() + ", segretaria=" + segretaria;
    }
}
```

Overriding e ereditarietà

L'overriding è strettamente legato all'ereditarietà:

- **la superclasse definisce un comportamento generale;**
- **la sottoclasse lo specializza per il proprio contesto.**

Riprendendo l'esempio precedente:

- `Impiegato` **definisce un metodo** `incrementaSalario()` **generico;**
- `Manager`, essendo un tipo particolare di impiegato, **può ridefinire lo stesso metodo per applicare una logica diversa (ad esempio un aumento maggiore o con criteri differenti).**

In questo modo, la classe `Manager` **evolve** il comportamento ereditato, senza duplicare codice.

✓ Regole per un overriding valido

Affinché l'overriding sia corretto, devono essere rispettate **tutte** le seguenti condizioni:

- **stesso nome del metodo**
- **stessa lista di parametri (numero, ordine e tipo)**
- **stesso tipo di ritorno**
  - sono ammessi anche **tipi covarianti**, cioè sottotipi del tipo di ritorno originale

⚠ Cosa NON si può fare in overriding

È importante chiarire cosa **non rientra** nel concetto di overriding:

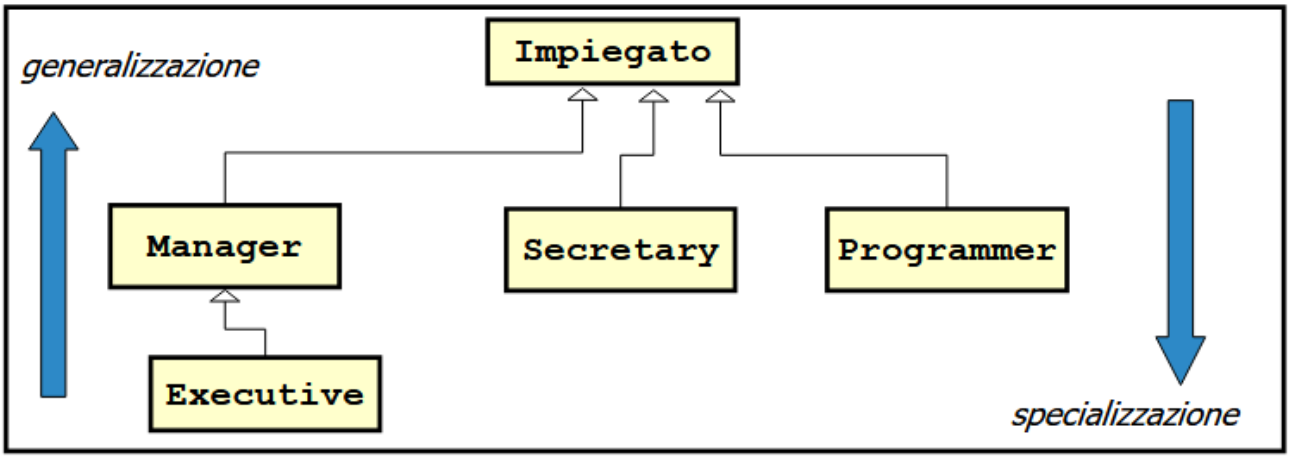
- ❌ **Gli attributi non si possono sovrascrivere**
  L'overriding riguarda solo i metodi, perché il polimorfismo in Java si basa sul comportamento, non sui dati.
- ❌ **I costruttori non possono essere sovrascritti**
  I costruttori **non partecipano all'ereditarietà**:
  - **non si fa overriding dei costruttori;**
  - **si parla invece di overloading (più costruttori nella stessa classe).**
- ❌ **I metodi** `static` **non supportano l'overriding**
  I metodi statici **appartengono alla classe, non agli oggetti.**
  - S**e una sottoclasse dichiara un metodo statico con lo stesso nome di uno della superclasse, si parla di ridefinizione (method hiding), non di overriding**.

- Il metodo chiamato dipende dal **tipo del riferimento**, non dall'oggetto reale.
  Se una di queste condizioni non è rispettata, **non si parla di overriding**, ma di un metodo diverso.

Gerarchia dell'ereditarietà

L'ereditarietà in Java **non è limitata a un solo livello** e può essere applicata più volte, permettendo di costruire **vere e proprie gerarchie di classi**.

Una gerarchia di ereditarietà descrive come le classi sono organizzate secondo relazioni di **generalizzazione e specializzazione**, e può svilupparsi in due direzioni principali:



L'immagine mostra chiaramente la distinzione tra **ereditarietà orizzontale** e **verticale**:

Ereditarietà orizzontale

Si parla di **ereditarietà orizzontale** quando:

- **più classi derivano dalla stessa superclasse.**

In questo caso:

- **la superclasse rappresenta un concetto generale;**
- **le sottoclassi rappresentano specializzazioni diverse dello stesso concetto.**

Esempio concettuale:

- `Impiegato`
  - `Manager`
  - `Segretaria`

    - `Programmatore`

Tutte le sottoclassi:

- **ereditano** struttura e comportamento comuni dalla superclasse;
- **aggiungono** o **specializzano** ciò che le distingue.

Questo tipo di ereditarietà è molto utile per modellare **insiemi e sottoinsiemi di oggetti**, in linea con la relazione *is-a* vista in precedenza.

Ereditarietà verticale

Si parla invece di **ereditarietà verticale** quando:

- **una classe derivata viene a sua volta estesa da un'altra classe, creando una catena di ereditarietà.**

In questo caso:

- **ogni livello della gerarchia aggiunge un grado ulteriore di specializzazione;**
- **il comportamento viene progressivamente raffinato.**

Esempio concettuale:

- `Impiegato`
    - `Manager`
        - `Executive`

Qui:

- `Manager` è una specializzazione di `Impiegato`;
- `Executive` è una specializzazione ulteriore di `Manager`.

> **✏ Nota didattica:**
>
> **Questi schemi consentono di massimizzare il riuso del codice, evitando duplicazioni, e di rappresentare chiaramente la progressiva specializzazione degli oggetti nel dominio applicativo**.
> L'ereditarietà orizzontale organizza concetti simili a uno stesso livello, mentre quella verticale permette di costruire **catene di evoluzione e raffinamento delle classi**

La classe cosmica: `Object`

In Java, come in Python e in JavaScript **tutte le classi ereditano per default dalla classe padre** `Object`, fatta eccezione per `Object` stessa.

La classe `java.lang.Object` rappresenta quindi la **radice di tutte le classi**:

- È la **superclasse universale**: **ogni classe in Java, direttamente o indirettamente, discende da** `Object`**.**
- Non possiede attributi propri, **ma fornisce alcuni metodi fondamentali utilizzabili da tutte le classi**:

Java

```java
public String toString()
public boolean equals(Object obj)
public void finalize()
```

Perché è importante:

- **Garantisce che tutti gli oggetti in Java condividano un comportamento minimo comune, rendendo possibile il polimorfismo**.
- **Grazie a** `Object`**, possiamo trattare qualsiasi oggetto come istanza di un tipo generico, ad esempio nelle collezioni (** `ArrayList<Object>` **)**.
- **I metodi ereditati da** `Object` **possono essere ridefiniti (overridden) nelle sottoclassi per fornire comportamenti specifici:**
  - come ad esempio una rappresentazione testuale personalizzata con `toString()` o un confronto tra oggetti con `equals()`.

> ⓘ **Nota didattica:**
>
> anche quando non definiamo esplicitamente una superclasse per le nostre classi, Java collega automaticamente ogni classe a `Object`, garantendo **coerenza e uniformità** nel modello di oggetti.

La classe `Object` in Java

Quindi tutte le classi in Java, tranne `Object` stessa, **ereditano implicitamente dalla classe** `java.lang.Object`**, che è quindi la radice dell'intera gerarchia di classi.**

La classe `Object` **non possiede attributi**, ma mette a disposizione alcuni metodi fondamentali che tutti gli oggetti Java ereditano:

1. `public String toString()` :
   - Restituisce una rappresentazione testuale dell'oggetto, del tipo:

```
NomePackage.NomeClasse@hashcode
```

- dove `hashcode` :
  - **indica un identificativo univoco dell'oggetto (non il valore effettivo degli attributi).**
- Per rendere questo metodo significativo, **spesso nelle sottoclassi si sovrascrive (`override`) per stampare informazioni utili sui dati dell'oggetto**.

2. `public boolean equals(Object obj)`
   - **confronta due oggetti restituendo `true` solo se il riferimento dell'oggetto chiamante è lo stesso dell'oggetto passato come parametro**.
   - **Anche questo metodo è tipicamente ridefinito nelle sottoclassi per confrontare il contenuto degli oggetti piuttosto che i loro riferimenti**.
3. `public void finalize()`
   - **Viene invocato dal Garbage Collector prima di deallocare la memoria di un oggetto**.
   - **Serve per rilasciare risorse esterne eventualmente allocate (file, connessioni di rete, ecc.).**
   - Nella pratica moderna, l'uso di `finalize()` è sconsigliato a favore di meccanismi più sicuri di gestione delle risorse, come `try-with-resources` .

> **Nota:**
>
> la presenza di `Object` come superclasse universale è ciò che rende possibili operazioni generiche su qualsiasi oggetto Java e costituisce la base per concetti come **polimorfismo** e **overriding dei metodi**.

Binding dinamico in Java

Il **binding** è la fase in cui il linguaggio stabilisce:

- **a quale funzione/metodo corrisponde una determinata invocazione.**

In Java, e anche in Python e JavaScript, il **binding è dinamico**, ovvero viene **risolto a run-time**:

- **Quando viene chiamato un metodo su un oggetto, la Java Virtual Machine (JVM) determina in quel momento quale implementazione del metodo deve essere eseguita.**
- Questo comportamento è essenziale per l'**ereditarietà e il polimorfismo**, perché permette a un oggetto di una sottoclasse di **sostituire** (override) metodi della superclasse e farli eseguire al posto della versione ereditata, anche se il riferimento è di tipo generico.
  Esempio concettuale:

Java

```java
Impiegato imp = new Manager("Luca", 2500.0, new Date(), "Giulia");
imp.incrementaSalario(500);
```

Anche se il tipo del riferimento è `Impiegato`, grazie al **binding dinamico**, la JVM eseguirà il metodo `incrementaSalario` della classe `Manager`, perché l'oggetto reale a run-time è un `Manager`.

Binding statico

- Nei linguaggi compilati tradizionali (es. C), il binding avviene **a compile-time**.
- Il compilatore decide **prima dell'esecuzione** quale funzione chiamare.
- **Vantaggi**: **esecuzione più veloce, controllo a compile-time.**
- **Svantaggi**: **minore flessibilità e impossibilità di sfruttare pienamente il polimorfismo.**

In sintesi:

il binding dinamico di Java permette al polimorfismo di funzionare correttamente, garantendo ch **e il comportamento effettivo di un metodo**

sia determinato dall'oggetto reale e non dal tipo del riferimento con cui lo si richiama.

Invocazione dei metodi e ricerca nella gerarchia di classi

Quando in Java si invoca un metodo su un oggetto, il compilatore e la JVM devono stabilire **quale metodo eseguire**.
Questo processo non si limita a guardare solo la classe in cui è stato dichiarato l'oggetto, ma prende in considerazione tutta la gerarchia di classi di cui l'oggetto fa parte.

Il funzionamento è il seguente:

1. **Verifica iniziale nella classe dell'oggetto**
   - **La JVM controlla se la classe che ha istanziato l'oggetto possiede un metodo con il nome e la lista di parametri specificata nella chiamata.**
   - **Se trova un metodo corrispondente, lo utilizza immediatamente.**
   - **Questo permette alle sottoclassi di fornire implementazioni personalizzate dei metodi ereditati dalla superclasse (overriding**).
2. **Ricerca nelle superclassi**
   - **Se la classe dell'oggetto non contiene il metodo richiesto, la JVM prosegue la ricerca nella superclasse immediata.**
   - **Questo processo continua risalendo lungo la catena di ereditarietà fino a trovare un metodo compatibile o fino a raggiungere la classe `Object`.**
   - **In questo modo, anche metodi definiti solo nelle classi superiori possono essere invocati dagli oggetti delle sottoclassi.**
3. **Esecuzione del metodo trovato**
   - **Quando la JVM individua un metodo valido, questo viene eseguito.**
   - **Se la sottoclasse ha ridefinito un metodo della superclasse, verrà eseguita la versione della sottoclasse, anche se il tipo di riferimento è quello della superclasse.**
   - Questo comportamento è alla base del **polimorfismo**:
     - **lo stesso nome di metodo può comportarsi in modo diverso a seconda della classe effettiva dell'oggetto.**

🔥 Nota importante:

Il compilatore controlla solo che il metodo sia dichiarato correttamente in qualche classe della gerarchia; la scelta definitiva di quale versione eseguire avviene a run-time, grazie al meccanismo del binding dinamico.

In sintesi, la JVM assicura che ogni invocazione di metodo sia coerente con la gerarchia delle classi e che il comportamento eseguito sia sempre quello corretto per il tipo reale dell'oggetto, non solo per il tipo dichiarato della variabile di riferimento.

Virtual Method invocation

La **Virtual Method Invocation** è un meccanismo gestito dalla **Java Virtual Machine (JVM)** che determina:

- **quale versione di un metodo sovrascritto deve essere effettivamente eseguita, basandosi sul tipo reale dell'oggetto e non sulla sua reference dichiarata.**

Questo significa che:

- **anche se una variabile è dichiarata come tipo di superclasse, la JVM eseguirà il metodo della sottoclasse, se quest'ultima ha fornito una nuova implementazione (overriding).**

Esempio pratico

Supponiamo di avere due classi:

Java

```java
class Impiegato {
    public void incrementaSalario(double importo) {
        System.out.println("Salario impiegato incrementato di: " + importo);
    }
}

class Manager extends Impiegato {
    @Override
    public void incrementaSalario(double importo) {
```

```
10          System.out.println("Salario manager incrementato di: " + (importo + 50));
11      }
12  }
```

Ora consideriamo il seguente codice:

```java
Impiegato imp = new Manager(); // 1
imp.incrementaSalario(100);    // 2
```

Cosa succede passo passo:

1. **Dichiarazione e istanza**

```java
Impiegato imp = new Manager();
```

- La reference `imp` è di tipo `Impiegato`.
- L'oggetto creato è di tipo `Manager`.
- Il compilatore verifica che `Impiegato` abbia il metodo `incrementaSalario`, quindi la riga è corretta.

2. **Invocazione del metodo**

```java
imp.incrementaSalario(100);
```

- Il compilatore sa solo che `imp` è un `Impiegato` e che il metodo `incrementaSalario` esiste, quindi compila senza errori.
- Durante l'esecuzione, la JVM legge il tipo reale dell'oggetto (`Manager`).
- Cerca il metodo `incrementaSalario` nella classe `Manager`.
- Lo trova e lo esegue, **ignorando la reference di tipo** `Impiegato`.
  Output atteso:

```yaml
Salario manager incrementato di: 150.0
```

🎟 Concetto chiave:

Il metodo invocato dipende dal tipo reale dell'oggetto, non dal tipo della variabile che lo contiene.
Questo comportamento è la base del **polimorfismo in Java**:

- **lo stesso nome di metodo può produrre comportamenti differenti a seconda della classe concreta dell'oggetto**.

📋 Binding Dinamico vs Virtual Method Invocation (VMI)

Finora abbiamo analizzato separatamente il **binding dinamico** e la **Virtual Method Invocation (VMI)**.
Poiché i due concetti sono strettamente collegati, è naturale chiedersi:

- Qual è la loro differenza?
- In che modo interagiscono tra loro?

1. **Binding dinamico:**
   - È il **principio secondo cui la JVM decide a run-time quale implementazione di un metodo eseguire.**

- Il collegamento tra la **chiamata al metodo e la sua implementazione concreta viene stabilito durante l'esecuzione del programma, non in fase di compilazione.**
- Senza binding dinamico:
- **il compilatore dovrebbe determinare in anticipo quale metodo invocare;**
- **l'overriding non produrrebbe effetti reali;**
- **il polimorfismo sarebbe solo apparente.**

Il binding dinamico è quindi la **condizione necessaria** per l'esistenza del polimorfismo in Java.

2. **Virtual Method Invocation (VMI)**

- **È il meccanismo operativo utilizzato dalla JVM quando un metodo viene invocato su un oggetto.**
- Consiste nel:

1. **leggere il tipo reale dell'oggetto a run-time;**
2. **cercare nella sua classe l'implementazione più specifica del metodo;**
3. **invocare quella versione.**

- La VMI **si basa sul binding dinamico**, poiché utilizza le informazioni disponibili a run-time per effettuare la scelta corretta.

> **Come lavorano insieme**
> - **Il binding dinamico stabilisce *che* la scelta del metodo avviene a run-time.**
> - **La Virtual Method Invocation è *come* la JVM realizza concretamente quella scelta.**

Regola D'Oro

- **Il binding dinamico** è:
  - **il *principio teorico* secondo cui l'associazione tra chiamata di metodo e implementazione concreta viene stabilita a run-time, in base al tipo reale dell'oggetto**.
- **La Virtual Method Invocation (VMI)** è:
  - **il *meccanismo operativo della JVM* che realizza concretamente quel principio quando un metodo viene invocato su un oggetto.**

In altre parole:

> Il binding dinamico è la regola teorica;
> la Virtual Method Invocation è il modo in cui la JVM utilizza per applicare quella regola.

In sintesi

2. **Overriding**:
   La **classe figlia** ridefinisce un metodo della superclasse, modificandone il comportamento.

3. **Reference di tipo padre**:
   Quando dichiari una variabile come tipo della superclasse e le assegni un oggetto della sottoclasse, ad esempio:

```java
Impiegato imp = new Manager();

```

qui `imp` è **di tipo Impiegato**, ma il **tipo reale dell'oggetto è Manager**.

4. **Invocazione del metodo**:
   Quando chiami il metodo overridato:

```java
imp.incrementaSalario(100);

```

- **Il compilatore verifica staticamente che `Impiegato` abbia il metodo `incrementaSalario`.**
- **A run-time, la JVM usa il binding dinamico e la virtual method invocation per determinare quale implementazione eseguire.**

4. **Risultato**:
   La JVM vede che il **tipo reale dell'oggetto** è `Manager`, quindi invoca la versione overridata del metodo, non quella originale della superclasse.

> ✓ In altre parole: **il tipo della reference serve solo per la compilazione, mentre il tipo reale dell'oggetto decide quale metodo viene eseguito a run-time.**

Polimorfismo

Il **polimorfismo** è una delle caratteristiche fondamentali della programmazione orientata agli oggetti:

- **indica la capacità di un oggetto di assumere molte forme, cioè di appartenere a più categorie**.

In Java, il polimorfismo è una **diretta conseguenza dell'ereditarietà**:

- **grazie alla possibilità di definire classi figlie che estendono e modificano il comportamento delle classi padre, un oggetto può essere trattato in modi diversi a seconda del contesto in cui viene utilizzato.**

I principali vantaggi del polimorfismo sono:

1. **Uniformità di riferimento**:
   - **È possibile utilizzare la stessa reference (di tipo generico, come una superclasse) per riferirsi a oggetti di tipi diversi.**
2. **Comportamento variabile**:
   - **Lo stesso metodo può essere invocato su oggetti di natura diversa, e il comportamento effettivo dipenderà dal tipo reale dell'oggetto**.
   - Questo è possibile grazie all'**overriding** dei metodi nelle sottoclassi e alla **virtual method invocation** della JVM, che a run-time seleziona automaticamente la versione corretta del metodo da eseguire.
3. **Flessibilità e manutenibilità**:
   - Non è necessario conoscere in anticipo tutti i tipi di oggetti che verranno gestiti:
     - **la chiamata al metodo è polimorfica, cioè si adatta dinamicamente al tipo reale dell'oggetto.**

In pratica, il polimorfismo permette di scrivere codice più **generico, modulare e riutilizzabile**, riducendo le duplicazioni e rendendo il software più flessibile di fronte a nuove esigenze o estensioni future.

Esempio: la classe Azienda

Immaginiamo di avere una classe `Azienda` che gestisce un **elenco di oggetti Impiegato** tramite una lista chiamata `elencoDip`.

Supponiamo che `Azienda` debba applicare un aumento di stipendio a tutti gli impiegati presenti nella lista.
In codice, possiamo fare un semplice **loop sulla lista** e invocare il metodo `incrementaSalario` su ciascun oggetto:

Java

```java
for (Impiegato imp : elencoDip) {
    if (imp != null) {
        imp.incrementaSalario(100);
    }
}
```

Qui entra in gioco il **polimorfismo**:

- Alcuni oggetti nella lista potrebbero essere **sottoclassi di Impiegato**, come `Manager` o `Executive`, **che hanno ridefinito (`overriding`) il metodo `incrementaSalario` per aggiungere comportamenti specifici**.
- La **JVM**, grazie alla **virtual method invocation**, **determina a run-time quale versione del metodo invocare per ciascun oggetto, in base al suo tipo reale.**

Questo significa che:

- Non è necessario scrivere condizioni o controlli aggiuntivi per distinguere tra impiegati, manager o executive.
- Il comportamento corretto di `incrementaSalario` viene applicato **automaticamente** per ciascun oggetto, rispettando la sua natura specifica.

In pratica, questo esempio mostra come il **polimorfismo renda il codice più flessibile, modulare e facilmente estendibile**:

- **possiamo aggiungere nuove sottoclassi di `Impiegato` senza modificare il loop o la logica di gestione dei salari.**

Assegnare un oggetto a un riferimento più generico

Abbiamo già visto nell'esempio della VMI e binding dinamico che è possibile assegnare a una variabile (reference) di tipo **superclasse** un oggetto istanza di una **sottoclasse**.
Esempio:

Java

```java
Impiegato imp = new Manager(...);
```

In questo caso:

- `Manager` **è un** `Impiegato` → relazione *is-a*

- quindi l'assegnazione è lecita.

Cosa succede realmente?

- In memoria viene creato un oggetto di tipo **Manager**.
- La variabile `imp` **è una reference di tipo Impiegato.**
- **La reference punta comunque a un oggetto** `Manager`**.**

Quindi:

- **Tipo della reference** → `Impiegato`
- **Tipo reale dell'oggetto** → `Manager`

Regola fondamentale

Un riferimento può essere di tipo più generico **solo se quel tipo appartiene alla gerarchia dell'oggetto**.

✓ **Valido**

Java

```java
Impiegato imp = new Manager(...);
Object obj = new Manager(...);
```

⚠ **Non valido**

Java

```java
Manager m = new Impiegato(...); // ERRORE

```

Perché non ogni Impiegato è un Manager.

Effetto dell'uso di un tipo più generico

Quando si utilizza una reference di tipo più generale rispetto all'oggetto reale (ad esempio `Impiegato imp = new Manager(...)`), si verificano due effetti distinti.

1. Gli attributi specifici vengono “oscurati”

Gli attributi definiti solo nella sottoclasse:

- **non vengono eliminati dall’oggetto,**
- **ma non sono accessibili tramite una reference della superclasse.**

Il motivo è che:

- **il compilatore verifica gli accessi basandosi esclusivamente sul tipo della reference.**
  Esempio:

```java
imp.secretaryName;    // ERRORE
```

L’attributo `secretaryName` esiste nell’oggetto `Manager`, ma il compilatore vede `imp` come un `Impiegato`, e quindi non ne consente l’accesso.

2. I metodi specifici non sono invocabili

Allo stesso modo:

```java
imp.getSecretary();  // NON COMPILA
```

Il metodo `getSecretary()` non è dichiarato nella classe `Impiegato`, quindi il compilatore blocca l’invocazione.
Anche qui vale la stessa regola:

- **la verifica della disponibilità di un metodo avviene in base al tipo della reference, non al tipo reale dell’oggetto.**

> ⚠ **Attenzione: qui entra in gioco il binding dinamico**
>
> Se invece il metodo è dichiarato nella superclasse ed è stato **sovrascritto (override)** nella sottoclasse:
>
> ```java
> imp.incrementaSalario(100);
> ```

Allora accade quanto segue:

- ==il compilatore controlla che `Impiegato` possieda il metodo `incrementaSalario` ==
- a run-time, la JVM verifica il tipo reale dell'oggetto (che è `Manager`)
- **tramite binding dinamico e Virtual Method Invocation, viene eseguita la versione del metodo definita in `Manager`**

Quindi:

- **visibilità a compile-time → dipende dal tipo della reference**
- **implementazione eseguita a run-time → dipende dal tipo reale dell'oggetto**

Questo meccanismo costituisce la base del **polimorfismo per sottotipo**.

Possiamo scrivere codice che lavora su `Impiegato` , ma che funziona correttamente anche con:

- `Manager`
- `Programmatore`
- `Segretaria`
- qualunque altra sottoclasse

Senza dover introdurre controlli espliciti sul tipo dell'oggetto.

Ed è proprio questa separazione tra:

- ciò che è visibile staticamente,
- e ciò che viene deciso dinamicamente,

a rendere il polimorfismo uno strumento potente e sicuro nella progettazione orientata agli oggetti.

Tipo più specifico (Downcasting)

Abbiamo visto che è sempre possibile assegnare a una reference più generica un oggetto più specifico:

Java

```
Impiegato imp = new Manager(...);
```

Questa operazione è lecita perché un `Manager` **è un** `Impiegato` (*relazione is-a*).

Ma è possibile fare l'operazione inversa?

```java
Manager m1 = imp;    // NON COMPILA
```

Perché non compila?

Perché il compilatore considera `imp` come un oggetto di tipo `Impiegato`.

A compile-time non può garantire che l'oggetto referenziato sia effettivamente un `Manager`.
Per questo motivo l'assegnazione viene bloccata: sarebbe potenzialmente insicura.

Quando è possibile?

È possibile solo se **l'oggetto referenziato è realmente un** `Manager`.

In tal caso è necessario un **cast esplicito:**

- cioè un'operazione con cui si comunica al compilatore che si è consapevoli del tipo reale dell'oggetto:

```java
Manager m1 = (Manager) imp;    // CORRETTO (se imp punta a un Manager)
```

Questo processo si chiama **downcasting:**

- **si scende nella gerarchia verso un tipo più specifico.**

Attenzione: rischio a run-time

Se `imp` non punta realmente a un `Manager`, il codice compila ma genera un errore a run-time:

```java
```

```
ClassCastException

```

Esempio:

```java
Impiegato imp = new Impiegato(...);
Manager m1 = (Manager) imp;   // Compila, ma errore a run-time
```

✓ **Per evitare questo problema si utilizza il controllo con `instanceof`:**

```java
if (imp instanceof Manager) {
  Manager m1 = (Manager) imp;
}
```

Origine dell'oggetto: identità reale

Come fa la JVM a gestire correttamente questi tipi?

Ogni oggetto in Java:

- **conserva internamente l'informazione sulla classe concreta da cui è stato istanziato;**
- **tale informazione è immutabile;**
- **non dipende dal tipo della reference che lo sta puntando.**

In memoria abbiamo quindi una distinzione netta tra:

```sass
Reference (Impiegato)  →  Oggetto reale (Manager)
```

L'oggetto è un `Manager`, anche se viene referenziato come `Impiegato`.

L’oggetto “porta con sé” il proprio tipo

Ogni oggetto possiede internamente un riferimento alla propria **classe concreta**.

Questa informazione viene utilizzata dalla JVM per:

- **eseguire il binding dinamico;**
- **determinare quale metodo invocare (Virtual Method Invocation);**
- **verificare la correttezza dei cast;**
- **supportare il polimorfismo.**

È possibile leggere questa informazione tramite:

Java

```java
imp.getClass();
```

Esempio:

Java

```java
System.out.println(imp.getClass());
```

Output (esempio):

Plain text

```
class Manager
```

Concetto fondamentale

L’oggetto ha una **identità reale** che:

- non cambia nel tempo
- non dipende dal tipo della reference
- viene usata dalla JVM per:
  - scegliere il metodo corretto (VMI)
- verificare la validità dei cast
- supportare il polimorfismo

Collegamento logico

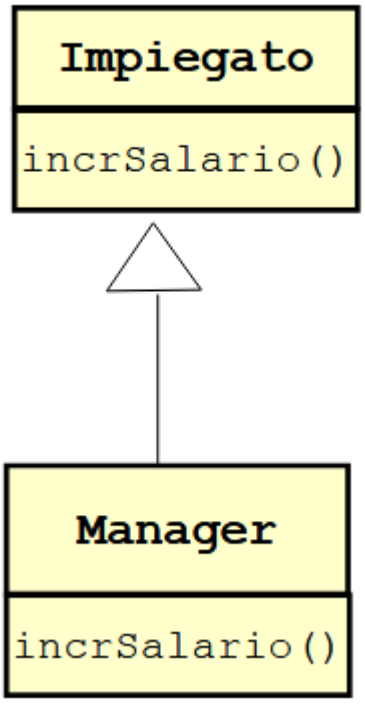
- La **reference** determina cosa è visibile a compile-time.
- L'**oggetto reale** determina quale comportamento viene eseguito a run-time.
- Il cast è una dichiarazione al compilatore:

> "So che questo oggetto è più specifico di quanto appare."

Tuttavia, la JVM verifica comunque a run-time che tale affermazione sia corretta.

Polimorfismo e overriding

Consideriamo la seguente gerarchia:



La classe `Manager` **sovrascrive** il metodo `incrSalario()` ereditato da `Impiegato`, fornendo una propria implementazione.

Ora analizziamo il seguente codice:

Java

```java
Impiegato imp = new Manager(...);
imp.incrSalario(800);
```

Quale metodo viene chiamato?

A prima vista potremmo pensare che venga chiamato il metodo di `Impiegato`, perché:

- la reference `imp` è di tipo `Impiegato`
- il compilatore controlla i metodi in base al tipo della reference

Infatti, a **compile-time**, il compilatore verifica che:

- `Impiegato` **possieda il metodo** `incrSalario()` ✓
- **quindi la chiamata sia formalmente corretta** ✓

Ma la decisione finale non viene presa in fase di compilazione.

Cosa accade a run-time?

Quando il metodo viene invocato, entra in gioco:

- il **binding dinamico**
- la **Virtual Method Invocation (VMI)**

La JVM:

1. **osserva il tipo reale dell'oggetto in memoria**
2. **verifica da quale classe è stato istanziato**
3. **cerca il metodo partendo da quella classe concreta**

Nel nostro caso:

- **l'oggetto è un** `Manager`
- **quindi la JVM cerca** `incrSalario()` **nella classe** `Manager`
- **lo trova**
- **e invoca la versione sovrascritta**

Quindi viene eseguito:

Java

```java
Manager.incrSalario()
```

Nonostante la reference sia di tipo `Impiegato`.

Principio chiave del polimorfismo

In Java:

- **la validità della chiamata dipende dal tipo della reference (compile-time)**
- **l'implementazione eseguita dipende dal tipo reale dell'oggetto (run-time)**

Questo è il cuore del **polimorfismo per sottotipo**:

possiamo trattare oggetti diversi come se fossero dello stesso tipo generale,
ma ciascuno si comporterà secondo la propria implementazione specifica.

Disabilitare l'ereditarietà

In Java è possibile impedire che una classe venga estesa, cioè che vengano create sottoclassi a partire da essa.

Per ottenere questo risultato si utilizza il modificatore **`final`** applicato alla dichiarazione della classe.

La sintassi è la seguente:

Java

```java
public final class NomeClasse {
}
```

Quando una classe viene dichiarata `final`, **il compilatore impedisce qualsiasi tentativo di estenderla.**
Di conseguenza, nessun'altra classe potrà utilizzare la parola chiave `extends` per derivare da essa.

Esempio concreto

Un esempio molto importante presente nella libreria standard di Java è la classe:

Java

```java
java.lang.String
```

La classe `String` è dichiarata `final`.
Questo significa che:

- **non è possibile creare una sottoclasse di `String`;**
- **non è possibile modificarne il comportamento tramite ereditarietà;**
- **il suo comportamento rimane garantito e immutabile dal punto di vista strutturale.**

Questa scelta progettuale è legata a motivi di sicurezza, coerenza e affidabilità.

Metodo `final` e overriding

Come gia sappiamo il modificatore `final` può essere applicato non solo alle classi, ma anche ai metodi.

Quando un **metodo** viene dichiarato `final`, significa che:

- **può essere ereditato dalle sottoclassi;**
- **non può essere ridefinito (overridden) in una sottoclasse.**

Esempio:

Java

```java
public class Impiegato {

    public final void calcolaStipendio() {
        // implementazione
    }
}
```

Se una classe `Manager` estende `Impiegato`, potrà utilizzare il metodo `calcolaStipendio()`, ma non potrà fornirne una nuova implementazione.

Se si tenta di fare overriding di un metodo dichiarato `final`, il compilatore genera un errore.

Differenza concettuale

È importante distinguere i due casi:

- **Classe `final` → impedisce l'ereditarietà.**
- **Metodo `final` → permette l'ereditarietà ma impedisce l'overriding.**

Nel primo caso si blocca la creazione di sottoclassi.
Nel secondo caso si consente la creazione di sottoclassi, ma si impedisce la modifica di uno specifico comportamento.

Conclusioni sul Polimorfismo

- **Il polimorfismo rappresenta una diretta conseguenza del meccanismo di ereditarietà.**
  Infatti, è proprio grazie alla relazione *is-a* tra superclassi e sottoclassi che **diventa possibile trattare oggetti di classi diverse attraverso un riferimento comune, tipicamente quello della superclass** e.
- **Attraverso il polimorfismo è possibile estendere e rendere più flessibili le applicazioni orientate agli oggetti con uno sforzo minimo** .
  Questo perché il codice può essere scritto facendo riferimento al tipo generale (superclasse o interfaccia), lasciando alle sottoclassi la possibilità di fornire implementazioni specifiche dei metodi tramite overriding.
- **Di conseguenza, per introdurre nuove classi o nuovi comportamenti, non è necessario modificare le classi già esistenti.**
  È sufficiente creare nuove sottoclassi che estendono quelle esistenti e ridefiniscono (se necessario) i metodi opportuni.

Questo principio è strettamente collegato al **principio Open/Closed** della programmazione orientata agli oggetti:

- le classi dovrebbero essere **aperte all'estensione**,
- ma **chiuse alla modifica**.

In questo modo si ottiene un sistema più modulare, manutenibile ed estendibile nel tempo, riducendo il rischio di introdurre errori nel codice già consolidato.